

AMOEBA DISTRIBUTED OPERATING SYSTEM: A CASE STUDY



SUBMITTED BY:

- 1. PRAJWAL THARU (21070021)**
- 2. PRATIKSHYA CHAND (21070022)**
- 3. PREM RAWAL (21070023)**
- 4. PUSHPA RAJ BHATTA (21070024)**
- 5. RAJ GHIMIRE (21070025)**

INTRODUCTION:

Amoeba is a general purpose distributed operating system. It is designed to take a collection of machines and make them act together as a single integrated system. In general, users are not aware of the number and location of the processors that run their commands, the number and location of file servers that store their files. It looks like a single old-fashioned time sharing system to a general user. An amoeba system can be thought of as a platform for doing research and development in distributed and parallel systems.

BACKGROUND:

Until about 1980, many organizations had a single large machine. The computers were thus huge and expensive. Later, as prices came down to a point where users could have their own personal computers, the machines were networked together which made users do remote login on other people's computers or share files in various ways possible.

Nowadays, some systems have many processors per user in the form of a parallel computer or a large collection of CPUs shared by a small group of users, which are referred to as parallel or distributed operating system. This development raised question of what kind of software will be needed for such systems. To devise a solution, a group under the direction of Prof. Andrew S. Tanenbaum at the Vrije University in Amsterdam developed a new distributed operating system called Amoeba.

SYSTEM ARCHITECTURE:

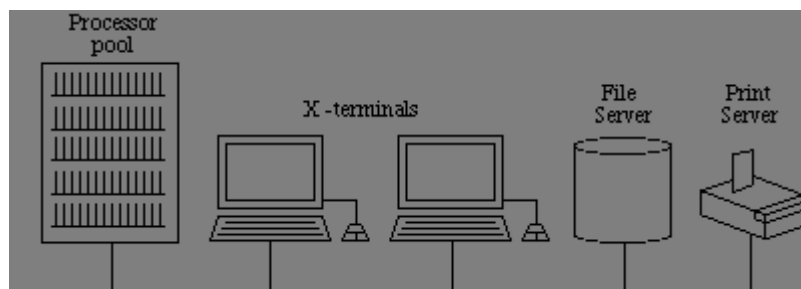


Fig: The Amoeba System Architecture

A. HARDWARE ARCHITECTURE:

A typical Amoeba system consists of three functional classes of machines:

- First, each user has a workstation for running the user interface, the X-window system.
- Second, there exists a pool of processors that are dynamically allocated to the users. These processors can be of different architecture.
- Third, there are specialized servers, such as file servers and directory servers that run all the time.

All these components must be connected by a fast LAN.

B. SOFTWARE ARCHITECTURE:

Amoeba consists of two basic pieces:

- Microkernel, which runs on every processor.
- A collection of servers that provide most of the operating system functionality.

1. The Amoeba Microkernel:

The microkernel has primary four functions:

- Manage processes and threads.
- Provide low level memory management support.

- Support communication.
- Handle low level I/O.

Let us discuss each of these in detail.

a. Process management:

Amoeba supports the concept of a process. It also supports multiple threads of control within single address space. Although a process with multiple threads has a single address space, each thread has its own register, program counter and a stack.

b. Low level memory management:

Threads can allocate and deallocate blocks of memory called segments. These segments can be read and written and can be mapped into and out of the address space of the process to which the calling thread belongs. A process must have at least one segment but it may have many more of them.

c. Inter-process communication:

Two forms of communications are provided: point to point communication and group communication. Point to point communication is based on the model of a client sending a message to a server and then blocking until the server has sent back a reply. Group communication allows a message to be sent from one source to multiple destinations.

d. Low level I/O management:

For each I/O device, there is a device driver in the kernel. These drivers manages all the I/O for the device.

2. The Amoeba Servers:

Amoeba is based on client server model. Everything that is not done by the kernel is done by the server processes thereby reducing kernel size and enhances flexibility.

PROCESS MANAGEMENT:

a. Basic Information:

A process in Amoeba is an address space and a collection of threads that run in it. When a process is created, the parent process is given a capability using which the child can be suspended, restarted, signaled or terminated. The children can create their own children, leading to a tree of processes.

b. Process handling levels:

In Amoeba, the process management is handled in three different levels.

- At the lowest level are the process servers, which are kernel threads running on every machine. To create a process on a given machine, another process does an RPC with that machine's process server, providing it with necessary information.
- At the next level, we have a set of library procedures that provide a more convenient interface for user programs. They do their job by calling the low level interface procedures.
- Finally, the simplest way to create a process is to use the run server, which does most of the work of determining where to run the new process.

c. Process descriptor:

Some process management calls use a data structure called process descriptor to provide information about a process to be run. One field tells which CPU architecture the process can be run on and the other field tells the process' owner's capability. When the process terminates or is stunned, RPC will be done using this capability to report the event. It contains descriptors for all the process' segments as well as descriptors for all its threads.

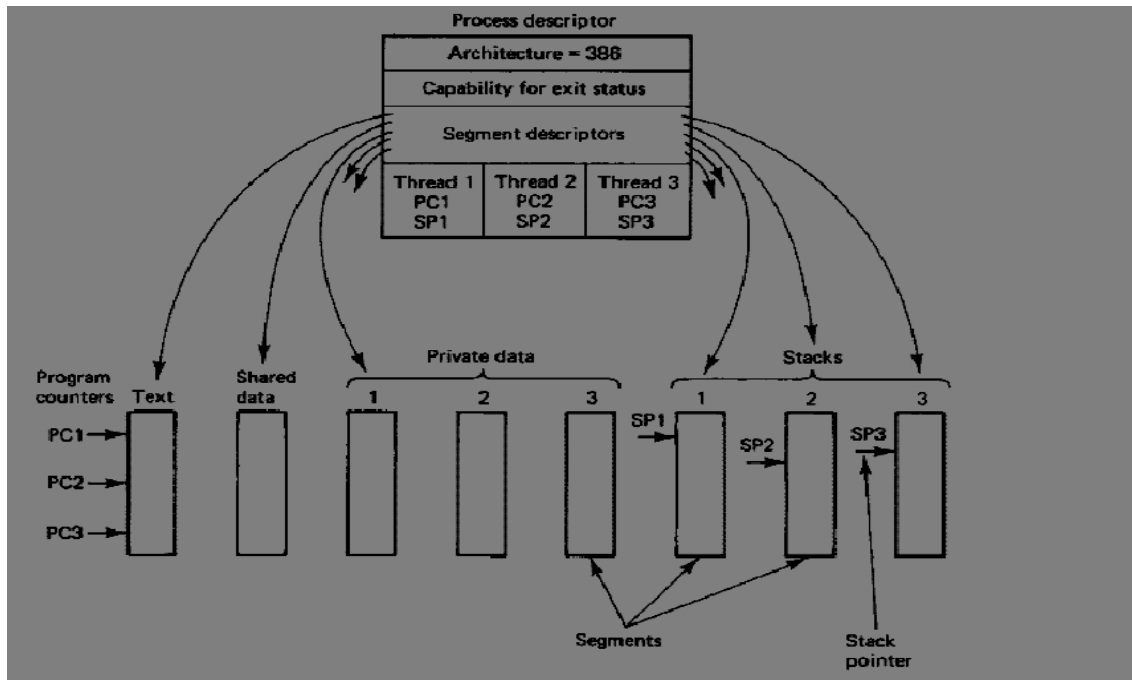


Fig.Process descriptor

d. Library procedures:

Low level process interface consists of following library procedures:

1. Exec: It's function is to do RPC with the specified process server asking to run the process.
2. Get load: It returns the information about the CPU speed, current load and the amount of memory free at the moment.
3. Stun: The parent can suspend a process by stunning it.

e. Threads:

Amoeba supports a simple thread model. When a process starts up, it has one thread. During execution, the process can create additional threads, and existing threads can terminate. The number of threads is therefore completely dynamic. When a new thread is created, the parameters to the call specify the procedure to run and the size of the initial stack. Although all threads in a process share the same program text and global data, each thread has its own stack, its own stack pointer, and its own copy of the machine registers. All the threads are managed by kernel.

Three methods are provided for threads to synchronize: signals, mutexes, and semaphores.

MEMORY MANAGEMENT:

a. Memory model:

The Amoeba memory model is basic. A process's address space is made up of segments mapped to virtual addresses. All segments must stay in memory while a process runs, without swapping or paging. This design was done for three reasons: performance, simplicity and economics. This simplicity boosts performance but restricts program size to fit within physical memory.

b. Segments:

A segment is a contiguous block of memory that contains code or data. Each segment has a capability that permits its holder to perform operations on it, such as reading or writing. Segments in the Amoeba memory model have an initial size that can change during process execution and may also be assigned

an initial value. These segments allow for reading and writing, making them suitable for constructing a main memory file server.

To create such a server, it begins by creating a segment as large as possible, determining this maximum size by querying the kernel. This segment serves as a simulated disk. The server then formats the segment as a file system, incorporating necessary data structures to manage files. Subsequently, it becomes operational, ready to handle client requests.

c. Mapped segments:

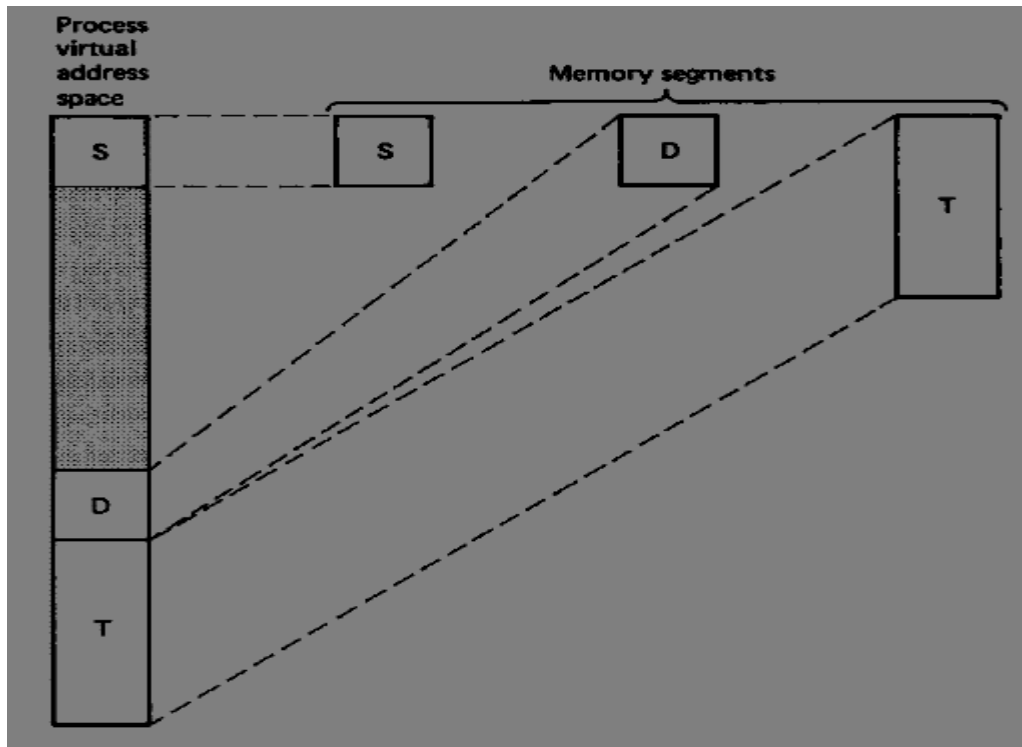


Fig. Process with three segments mapped into its virtual address space

- When a process is started, it must have at least one segment.
- Once it is running, a process can create additional segments and map them into its address space at any unused virtual address.
- A process can unmap segments, which means it can specify a range of virtual addresses and request that this range be removed.
- Once unmapped, those addresses are no longer valid for use. However, a capability is returned to the process, allowing it to still access the segment if needed. The segment can even be mapped back in later, possibly at a different virtual address.

COMMUNICATION MANAGEMENT:

Amoeba supports two forms of communication: remote procedure call using point to point message passing and a group communication.

a. RPC:

RPC takes place with client sending messages to the server and with server sending back the response. The RPC mechanism makes use of three principal kernel primitives:

1. get request: indicates server's willingness to listen to port
2. put reply: done by server when it has a reply to send
3. trans: send message from client to server and wait for reply

Security:

It is possible for an intruder to impersonate a server by just doing a get_request on server's port. Amoeba solves this problem cryptographically as-

- Each port is a pair of port: the get-port is known only to the server and the put-port is known to the outside world.
- These two ports are related through one-way function F as $\text{put-port} = F(\text{get-port})$

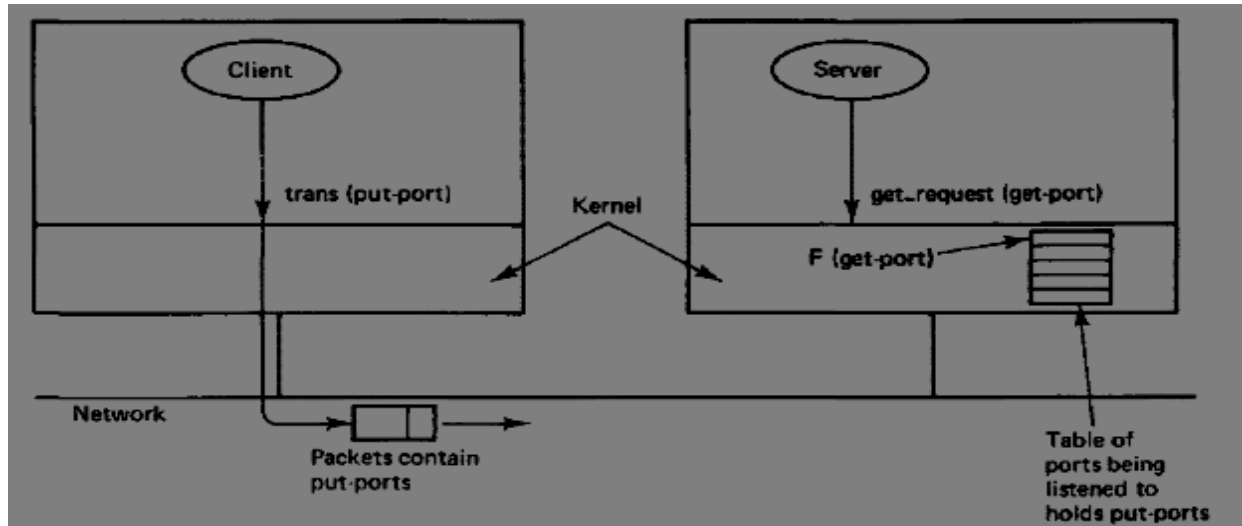


Fig: Relationship between get-port and put-port

b. Group communication:

Amoeba uses the concept of closed groups. Processes can join and leave the group dynamically and can be the members of multiple group at the same time.

The following table lists out the group communication primitives:

Call	Description
CreateGroup	Create a new group and set its parameters
JoinGroup	Make the caller a member of a group
LeaveGroup	Remove the caller from a group
SendToGroup	Reliably send a message to all members of a group
ReceiveFromGroup	Block until a message arrives from a group
ResetGroup	Initiate recovery after a process crash

Properties of group communication:

- It ensures ordered delivery of messages that is if two processes send messages to a group almost simultaneously, the system ensures that all the group members will receive the messages in same order.

- Amoeba can withstand the loss of an arbitrary collection of k (degree of resilience) processes.
- k is specified by user as a parameter for creating a group.
- The larger the value of k , more redundancy is required and slower the group communication becomes.

CHALLENGES:

The system faced scalability issues, requiring efficient handling of numerous nodes while maintaining performance. Ensuring fault tolerance was crucial to manage failures gracefully in a distributed environment. Resource management caused a challenge, requiring optimization of CPU, memory, and network utilization across nodes. Security measures had to be robust to protect against unauthorized access and malicious attacks.

PRESENT STATUS:

Currently, amoeba is being used for several practical situations from developing environment for parallel computing to industrial applications.

CONCLUSION:

In conclusion, Amoeba operating system showcases the impact of creative design and teamwork in shaping future of computing. It introduced revolutionary concepts in distributed operating systems, influencing how modern technology operates and paved the way for today's interconnected decent-realized operating system.